

Koneksi Springboot-MySQL

Preparation

- Spring web
- Spring Data JPA
- MySQL Driver
- Lombok

Konfigurasi Koneksi

- /src/main/resources/application.service

```
spring.datasource.url=jdbc:mysql://localhost:3306/db_kampus
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Model

- .../model/Mahasiswa.java

```
package com.example.javadb.model;
```

```
import jakarta.persistence.Column;  
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
import jakarta.persistence.Table;  
import lombok.AllArgsConstructor;  
import lombok.Data;  
import lombok.NoArgsConstructor;
```

Windsurf: Refactor | Explain

```
@Entity
```

```
@Table(name = "mahasiswa")
```

```
@Data
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```
public class Mahasiswa {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @Column(nullable = false, unique = true)
```

```
    private String nim;
```

```
    @Column(nullable = false)
```

```
    private String nama;
```

```
    private String jurusan;
```

```
    private Integer semester;
```

```
    private String email;
```

```
}
```

Repository

Bentuk umum dari Repository

JpaRepository<Entity, TipePrimaryKey>

```
package com.example.javadb.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.javadb.model.Mahasiswa;
```

Windsurf: Refactor | Explain

```
public interface MahasiswaRepository extends JpaRepository<Mahasiswa, Long> {  
}
```

Service

```
private final MahasiswaRepository mahasiswaRepository;

public MahasiswaService(MahasiswaRepository mahasiswaRepository) {
    this.mahasiswaRepository = mahasiswaRepository;
}
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

```
public List<Mahasiswa> getAllMahasiswa() {
    return mahasiswaRepository.findAll();
}
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

```
public Mahasiswa getMahasiswaById(Long id) {
    return mahasiswaRepository.findById(id)
        .orElseThrow(() -> new RuntimeException(message: "Mahasiswa tidak ditemukan"));
}
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

```
public Mahasiswa createMahasiswa(Mahasiswa mahasiswa) {
    return mahasiswaRepository.save(mahasiswa);
}
```

Controller

```
private final MahasiswaService mahasiswaService;  
  
public MahasiswaController(MahasiswaService mahasiswaService) {  
    | this.mahasiswaService = mahasiswaService;  
}
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

@GetMapping

```
public List<Mahasiswa> getAllMahasiswa() {  
    | return mahasiswaService.getAllMahasiswa();  
}
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

@GetMapping("/{id}")

```
public Mahasiswa getMahasiswaById(@PathVariable Long id) {  
    | return mahasiswaService.getMahasiswaById(id);  
}
```

Windsurf: Refactor | Explain | Generate Javadoc | ✕

@PostMapping

```
public Mahasiswa createMahasiswa(@RequestBody Mahasiswa mahasiswa) {  
    | return mahasiswaService.createMahasiswa(mahasiswa);  
}
```

Custom Repository Method

```
public interface MahasiswaRepository extends JpaRepository<Mahasiswa, Long> {  
  
    List<Mahasiswa> findByNama(String nama);  
  
    @Query(  
        value = """  
            SELECT *  
            FROM mahasiswa  
            WHERE nama LIKE %:keyword%  
        """,  
        nativeQuery = true  
    )  
    List<Mahasiswa> searchByNama(  
        @Param("keyword") String keyword  
    );  
}
```

Ada dua cara dalam custom ini
native query dan JPA Entity

dalam contoh method findByNama
JPA akan otomatis membuat
query Search by nama
yang Dimana nama adalah Entity
Field dalam Class Mahasiswa

Berbeda dengan contoh kedua,
yang murni menggunakan native
query

Tanda “""" adalah textblock (Java 15+)
Yang digunakan untuk penulis yang
lebih enak dibaca